Anjuman-I-Islam's

M. H. SABOO SIDDIK COLLEGE OF ENGINEERING

8, Saboo Saddik Polytechnic Road, Byculla, Mumbai, Maharashtra 400008

DEPARTMENT OF INFORMATION TECHNOLOGY

ITM601 MINI PROJECT - 2B

Title of the Project

House of Cards

Supervisor/Guide

ER SWATI GURAV

REV - 2019 'C' Scheme

University of Mumbai

Academic Year (2025 -26)

CERTIFICATE

This is to certify that the project entitled "**House Of Cards** by "**Yamin Khan" (242464),** "**Abdurrahman Qureshi" (242466)**, "**Mohammed Rumaan Shaikh" (242469)**, "**Abdul Sameer Shaikh" (242471)**" submitted to the University of Mumbai in partial fulfilment of the requirement for the ITM601 Mini Project - 2B for ML project of the 6TH Semester in **Department of Information Technology**.

(ER. SWATI GURAV)
SUPERVISOR/GUIDE

PROF. RASHEED NOOR
MINI PROJECT CO-ORDINATOR

(Dr. Zainab Mirza)
HEAD OF DEPARTMENT

MINI PROJECT REPORT APPROVAL

This project report entitled "**House of Cards** " by "**Yamin Khan" (242464),** "**Abdurrahman Qureshi" (242466)**, "**Mohammed Rumaan Shaikh" (242469)**, "**Abdul Sameer Shaikh" (242471)** is approved for the ITM601 Mini Project 2B for using ML project of the 6th Semester.

EXAMINER 1

EXAMINER 2

Date :

Place : Mumbai

ACKNOWLEDGEMENT

We would take this humble opportunity to express our sincere gratitude to our project guide ER. Swati Gurav for her valuable advice and direction under

Which the project was executed and to all respected people who provided their kind support and encouragement throughout the project. The success of this project to the most extent rests on every bit of suggestions, advice, and help which they provided us from time to time and also being open and constructive to all the problems that we faced during the entire development lifecycle of our project.

We would also like to extend a big vote of thanks to the coordinator, our teachers, for their constant support and encouragement

YAMIN KHAN (242464)

ABDURRAHMAN QURESHI (242466)

MOHAMMED RUMAAN SHAIKH (242469)

ABDUL SAMEER SHAIKH (242471)

ABSTRACT

The House of Cards project presents the design and development of a strategic, rule-based card game inspired by traditional games like Blackjack and Three Card Poker but enhanced with thematic elements and interactive gameplay mechanics. The primary objective of this system is to create an engaging multiplayer card game that combines numerical decision-making with strategic use of special-action cards, thereby increasing player involvement and replayability.

The game utilizes a custom deck of 100 cards, divided into 80 numerical cards (values 0–9 distributed across four houses: Gryffindor, Ravenclaw, Hufflepuff, and Slytherin) and 20 special cards (Echo, Mirror, Shield, Swap, and Hallows). Players aim to reach a predefined target sum without exceeding it, while strategically using special cards to influence gameplay, disrupt opponents, and gain advantages. The game supports both human players and AI-controlled bots, allowing for flexible gameplay modes including solo practice and multiplayer simulation.

The system is implemented using Python for core game logic and backend processing, with Flask acting as the web framework to handle communication between the frontend and backend. The user interface is developed using HTML, CSS, and JavaScript, ensuring an interactive and responsive gaming experience. Visual elements such as cards and themes are designed using Adobe Photoshop to enhance the overall aesthetics of the game.

A key feature of the system is the use of a heuristic rule-based model instead of machine learning. This model governs both the game engine and bot behavior through predefined rules and decision-making strategies. AI bots analyze the current game state, including the running sum and available cards, to make calculated moves such as minimizing risk near the target or maximizing advantage using special cards. This approach ensures deterministic behavior, ease of implementation, and efficient testing.

TABLE OF CONTENT

INTRODUCTION

Card games have been a popular form of entertainment and strategic thinking for decades, combining elements of probability, decision-making, and player interaction. Traditional games such as Blackjack focus primarily on numerical strategies, where players aim to reach a target value without exceeding it. While such games are simple and engaging, they often lack deeper interaction and strategic diversity beyond basic risk management.

The House of Cards project is developed to address these limitations by introducing a modern, themed card game that blends numerical gameplay with strategic special-action mechanics. The game is inspired by Blackjack but extends its core concept by incorporating unique card types, house-based themes, and interactive gameplay features. This results in a more dynamic and engaging experience where players must not only manage numbers but also strategically use special abilities to influence the outcome of the game.

The game uses a custom deck of 100 cards, divided into numerical cards (0–9) and special cards such as Echo, Mirror, Shield, Swap, and Hallows. These cards are distributed across four themed houses—Gryffindor, Ravenclaw, Hufflepuff, and Slytherin—adding a layer of identity and variation to the gameplay. Players take turns playing cards to a common pile, continuously updating a running sum, with the goal of reaching a predefined target without exceeding it. Special cards introduce additional complexity by allowing players to manipulate the game state, block opponents, or gain strategic advantages.

A key feature of the system is the inclusion of AI-controlled bots, which enable both single-player and multiplayer gameplay modes. These bots are designed using a heuristic rule-based approach, allowing them to make decisions based on the current game state rather than relying on machine learning. This ensures predictable, efficient, and testable behavior while still providing competitive gameplay.

From a technical perspective, the system is implemented using Python for backend logic, with Flask serving as the web framework to manage communication between the frontend and backend. The user interface is built using HTML, CSS, and JavaScript, ensuring an interactive and user-friendly experience. Visual elements are designed using Adobe Photoshop, enhancing the aesthetic appeal of the game.

Overall, the House of Cards project aims to demonstrate how traditional card game concepts can be enhanced through modern technologies and innovative design. By combining structured game logic, interactive UI, and strategic AI behavior, the project delivers a balanced and engaging card game system suitable for both learning and entertainment.

METHODOLOGY

The development of **House of Cards** follows a systematic and modular approach to ensure efficient design, implementation, and testing of the game system. The methodology focuses on defining clear rules, implementing structured game logic, integrating user interaction, and simulating intelligent gameplay using bots.

1. Game Design and Rule Definition

The first step involves designing the core concept of the game, including rules, objectives, and win conditions. A custom deck of 100 cards is defined, consisting of 80 numerical cards (0–9) and 20 special cards (Echo, Mirror, Shield, Swap, Hallows), distributed across four houses. The gameplay is structured around maintaining a running sum and reaching a target value without exceeding it.

2. Data Structure and Deck Management

Efficient data structures such as lists and objects are used to represent cards, player hands, personal decks, and discard piles. The deck is shuffled and distributed among players. Each player maintains a personal deck and a hand, while a central discard pile and leftover deck are used for round management.

3. Game Logic Implementation

The core game engine is implemented in Python , where all rules and conditions are enforced. The system manages:

- Turn-based gameplay in a clockwise manner
- Updating the current sum after each move
- Handling round transitions and reset conditions
- Checking win conditions when a player exhausts all cards

4. Special Card Handling

Each special card is implemented with specific logic to influence gameplay:

- Echo doubles the next number played
- Mirror repeats the last number card
- Shield cancels the effect of certain special cards
- Swap exchanges cards between hand and deck
- Hallows triggers an instant limit condition

Proper sequencing and conflict resolution are ensured when multiple effects interact.

5. Heuristic Bot Implementation

AI bots are developed using a heuristic rule-based model . Instead of learning from data, bots follow predefined strategies:

- Evaluate current sum and risk level
- Select safe or aggressive moves accordingly
- Use special cards strategically based on game context

This allows efficient simulation of intelligent opponents and supports single-player gameplay.

6. Frontend Integration

The user interface is developed using HTML, CSS, and JavaScript to provide an interactive experience. The frontend displays player cards, current sum, and game status, while capturing user inputs for gameplay actions.

7. Backend Integration using Flask

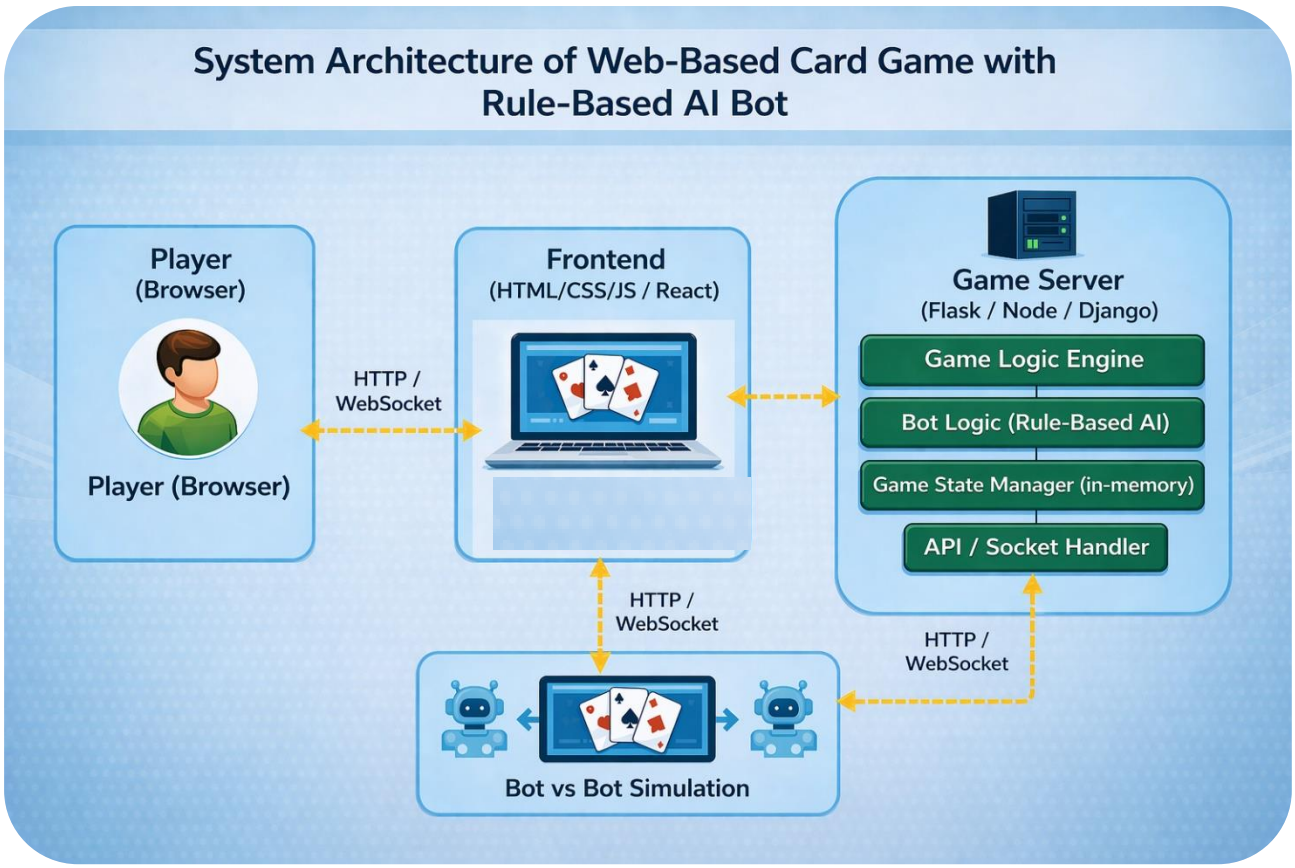
The Flask framework is used to connect the frontend with the backend logic. API routes are created to handle game actions, process moves, and return updated game states, enabling smooth communication between client and server.

8. Testing and Validation

The system is tested under multiple scenarios, including normal gameplay, edge cases, and special card interactions. Debugging and validation ensure that rules are correctly enforced and the gameplay remains fair and balanced.

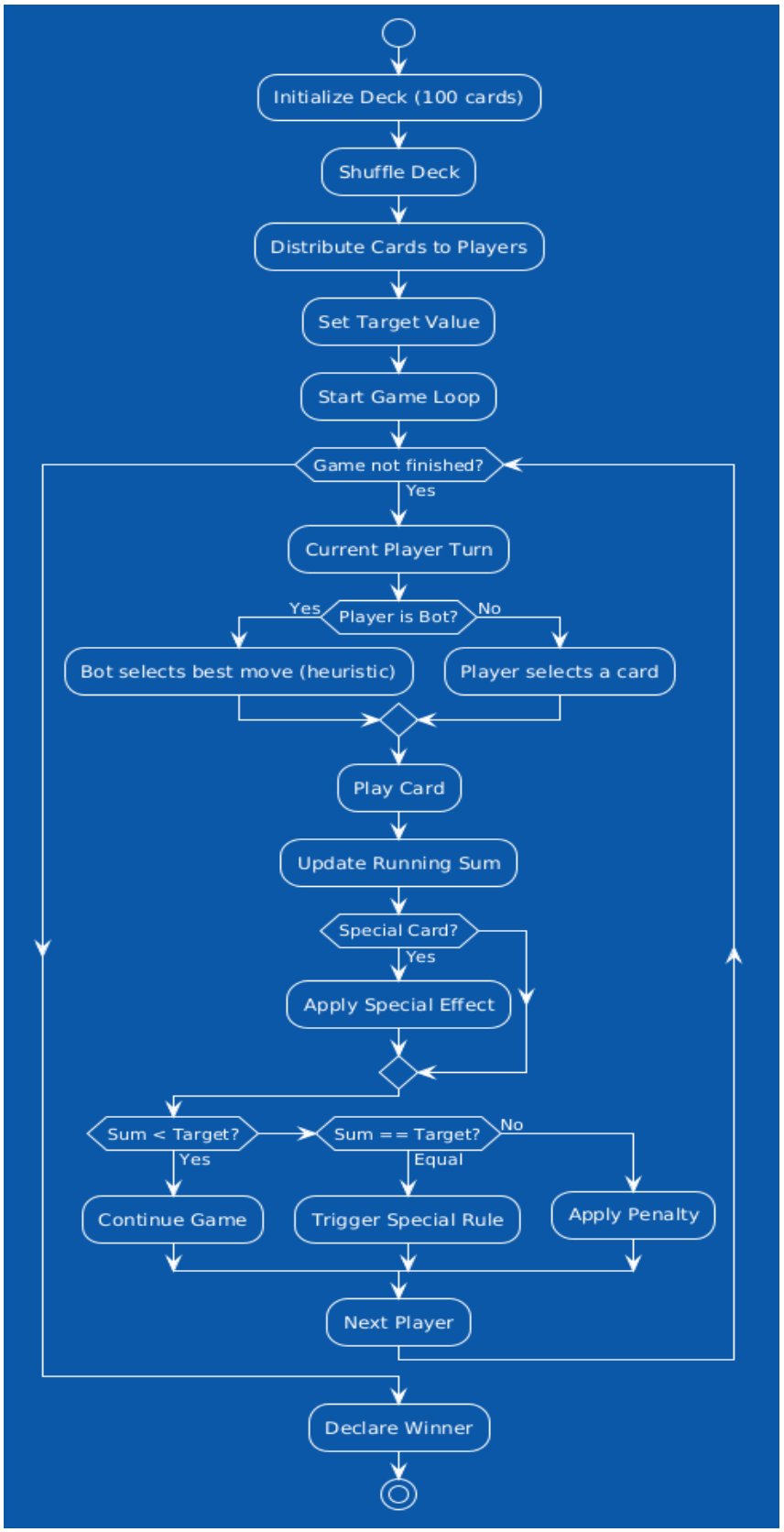
IMPLEMENTATION AND DESIGN

System Architecture

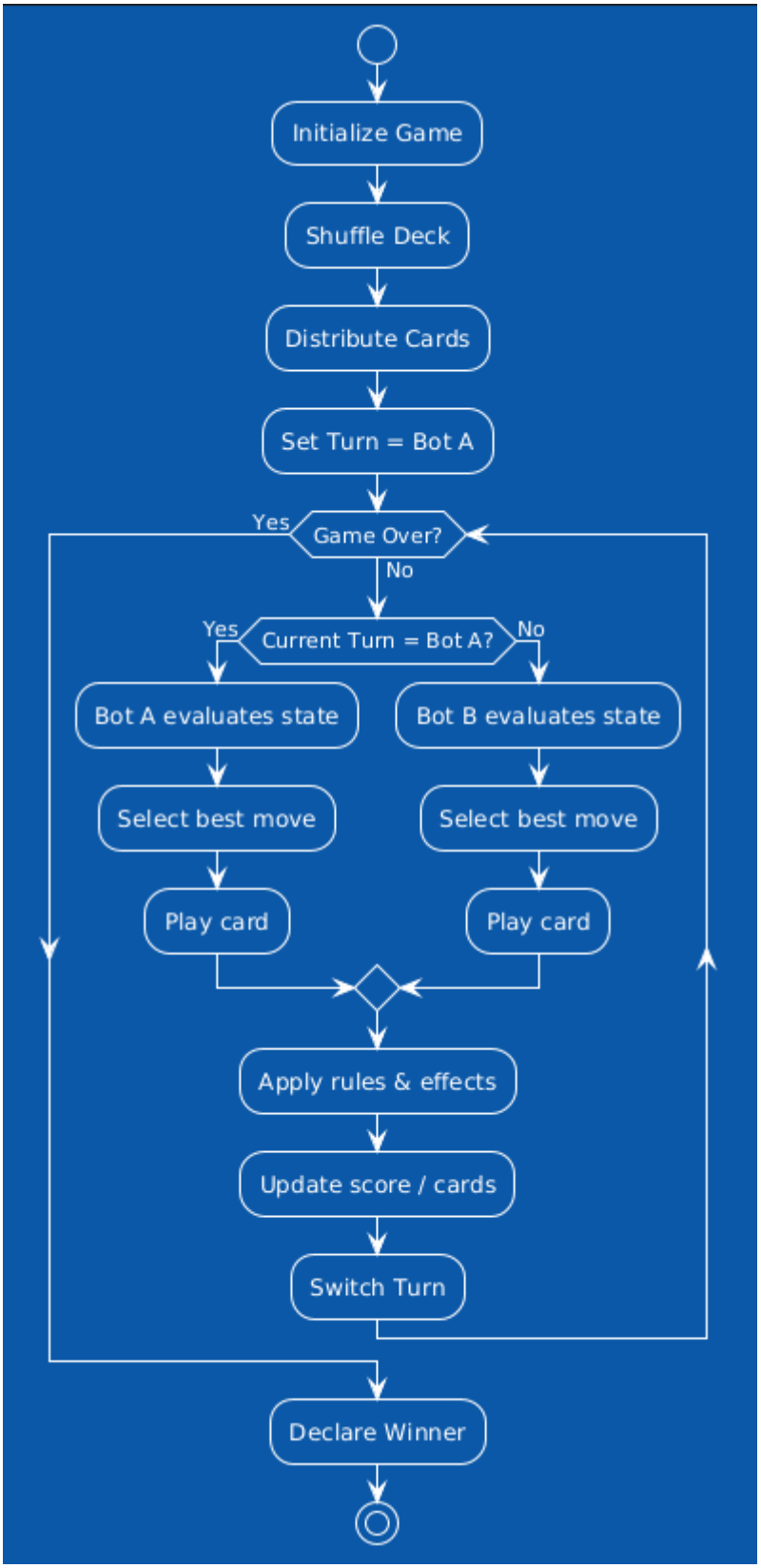


- Follows client-server architecture
- Frontend built using HTML, CSS, JavaScript
- Backend implemented using Flask (Python)
- Game logic handled by Python game engine
- API communication between frontend and backend
- Includes bot module for AI-based moves
- Ensures modularity and scalability

Flowchart: PvE (Player vs bot)



Flowchart: Simulation (Bot vs Bot)



TESTING /RESULT & ANAYLIS

The system is tested using multiple test cases to ensure correct implementation of game logic, special card effects, and bot behavior. The testing process verifies that the game runs smoothly under normal and edge conditions.

Test Cases:

Test Case ID	Test Scenario	Input	Expected Output	Result
TC01	Deck Initialization	Start game	100 cards created and shuffled properly	Pass
TC02	Card Distribution	Game start with players	Cards distributed equally among players	Pass
TC03	Normal Card Play	Play numerical card	Running sum updates correctly	Pass
TC04	Echo Card	Play Echo + number	Next number value is doubled	Pass
TC05	Mirror Card	Play Mirror	Last played number is added again	Pass
TC06	Shield Card	Use Shield on Echo/Mirror	Special effect is canceled	Pass
TC07	Swap Card	Play Swap	Card exchanged with deck	Pass
TC08	Hallows Card	Play Hallows	Target reached, others draw penalty	Pass
TC09	Sum Limit Condition	Sum exceeds target	Penalty applied	Pass
TC10	Bot Decision Making	Bot turn	Bot selects optimal move based on heuristic	Pass
TC11	Game Completion	Player finishes cards	Winner declared correctly	Pass

To ensure the correctness and performance of the **House of Cards** system, the following testing techniques were applied:

- **Unit Testing:**
  Unit testing was performed on individual components such as card distribution, sum calculation, and special card logic. Each module was tested independently to ensure correct functionality.

- **Integration Testing:**
  Integration testing was carried out to verify the interaction between different modules, including the frontend, Flask backend, and game engine. This ensured proper data flow and communication across the system.

- **Functional Testing:**
  Functional testing was used to validate that all features of the game, including gameplay mechanics and special card effects, work according to the defined rules and requirements.

- **Performance Testing:**
  Performance testing evaluated the responsiveness and stability of the system during continuous gameplay and bot interactions, ensuring smooth and efficient operation.

Result: Output Images

Fig.1

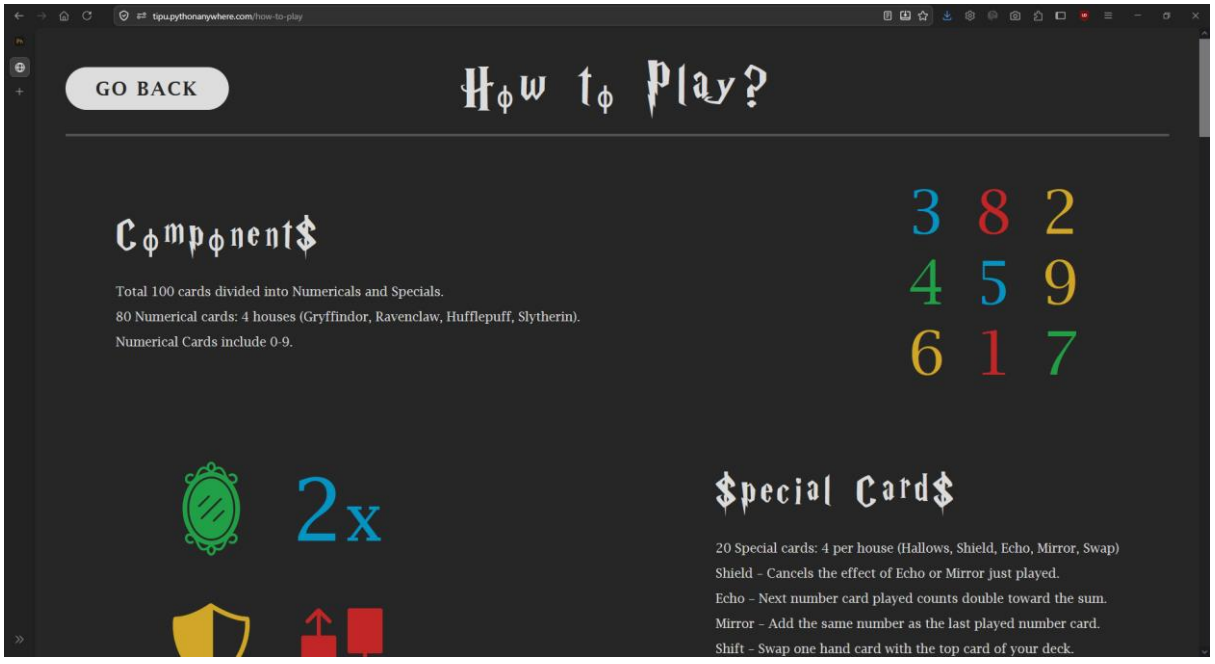


Fig.2

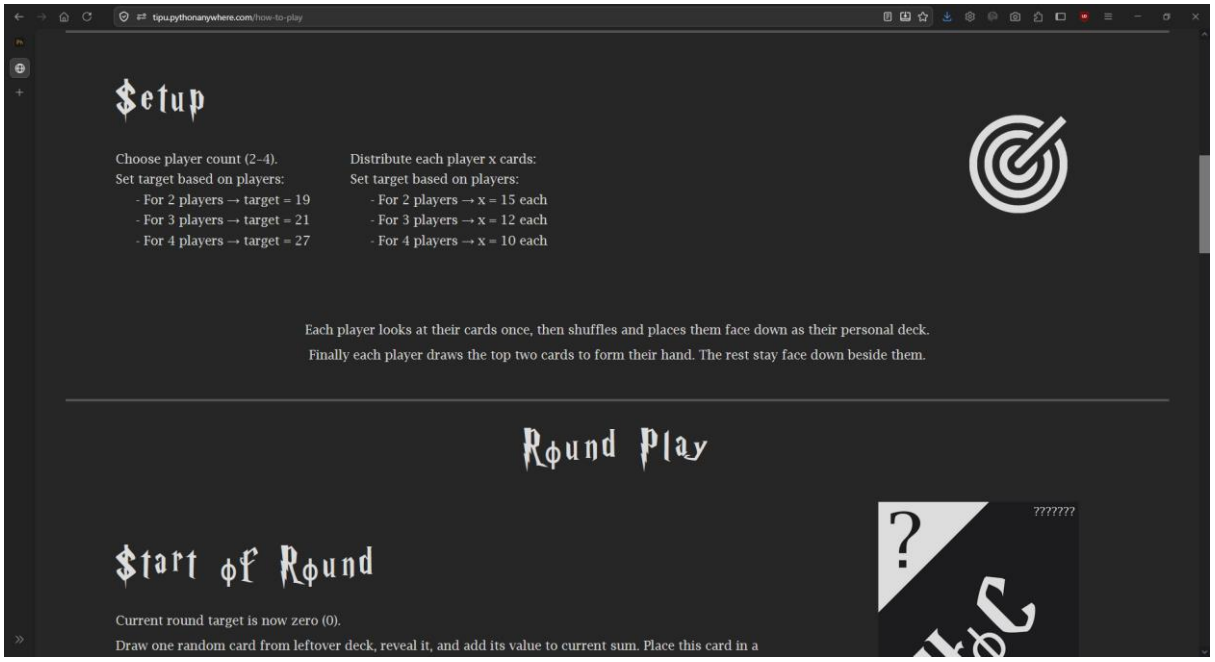

tipu.pythonanywhere.com/how-to-play
Setup
Choose player count (2–4).
Set target based on players:
- For 2 players → target = 19
- For 3 players → target = 21
- For 4 players → target = 27
Distribute each player x cards:
Set target based on players:
- For 2 players → x = 15 each
- For 3 players → x = 12 each
- For 4 players → x = 10 each
Each player looks at their cards once, then shuffles and places them face down as their personal deck.
Finally each player draws the top two cards to form their hand. The rest stay face down beside them.
Round Play
Start of Round
Current round target is now zero (0).
Draw one random card from leftover deck, reveal it, and add its value to current sum. Place this card in a

Fig.3

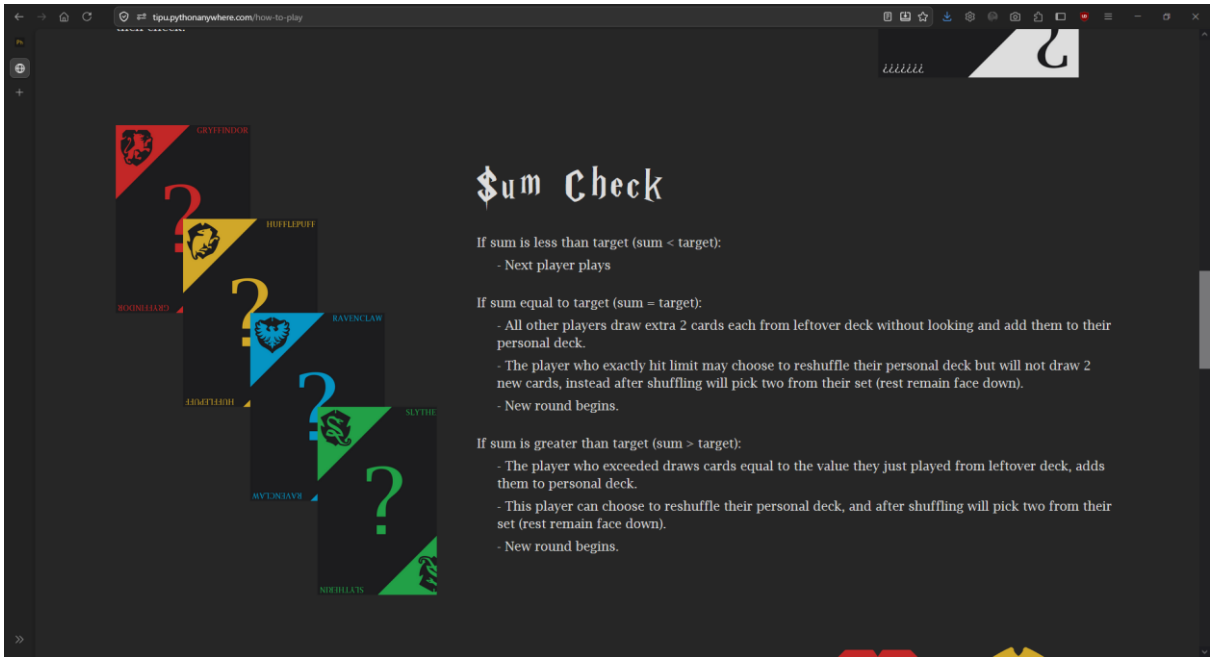

tipu.pythonanywhere.com/how-to-play
GRYFFINDOR
HUFFLEPUFF
RAVENCLAW
SLYTHE
Sum Check
If sum is less than target (sum < target):
- Next player plays
If sum equal to target (sum = target):
- All other players draw extra 2 cards each from leftover deck without looking and add them to their personal deck.
- The player who exactly hit limit may choose to reshuffle their personal deck but will not draw 2 new cards, instead after shuffling will pick two from their set (rest remain face down).
- New round begins.
If sum is greater than target (sum > target):
- The player who exceeded draws cards equal to the value they just played from leftover deck, adds them to personal deck.
- This player can choose to reshuffle their personal deck, and after shuffling will pick two from their set (rest remain face down).
- New round begins.

Fig.4

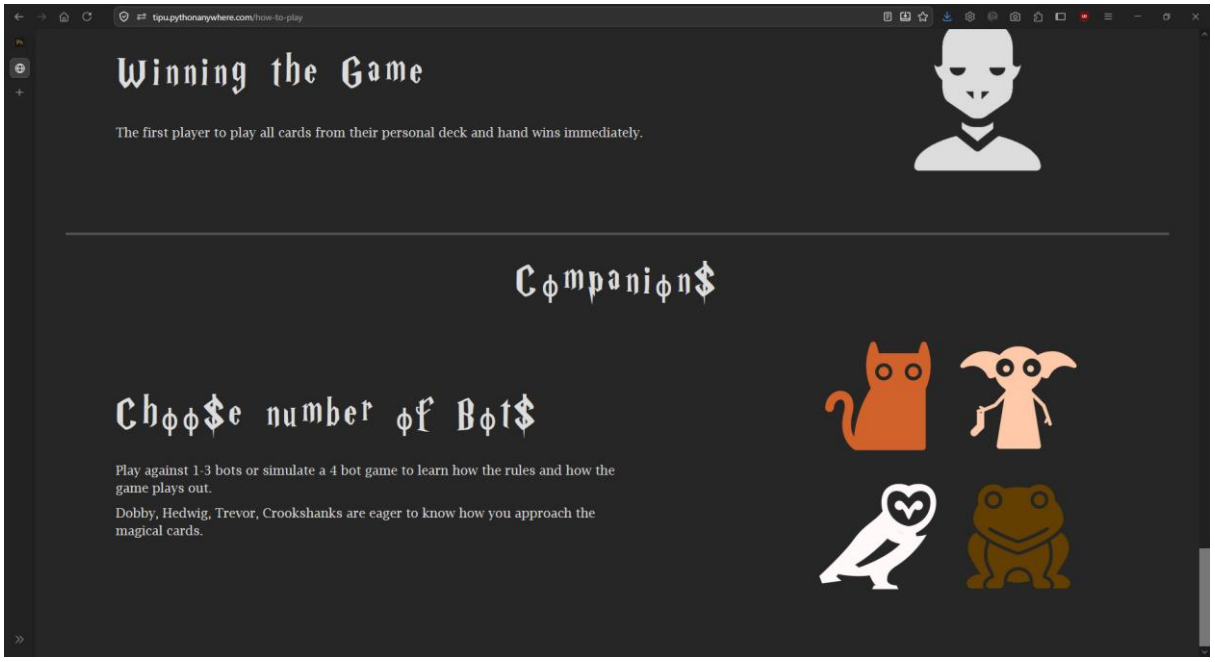

tipu.pythonanywhere.com/how-to-play
Winning the Game
The first player to play all cards from their personal deck and hand wins immediately.
Companions
Choose number of Bots
Play against 1-3 bots or simulate a 4 bot game to learn how the rules and how the game plays out.
Dobby, Hedwig, Trevor, Crookshanks are eager to know how you approach the magical cards.

Fig.5

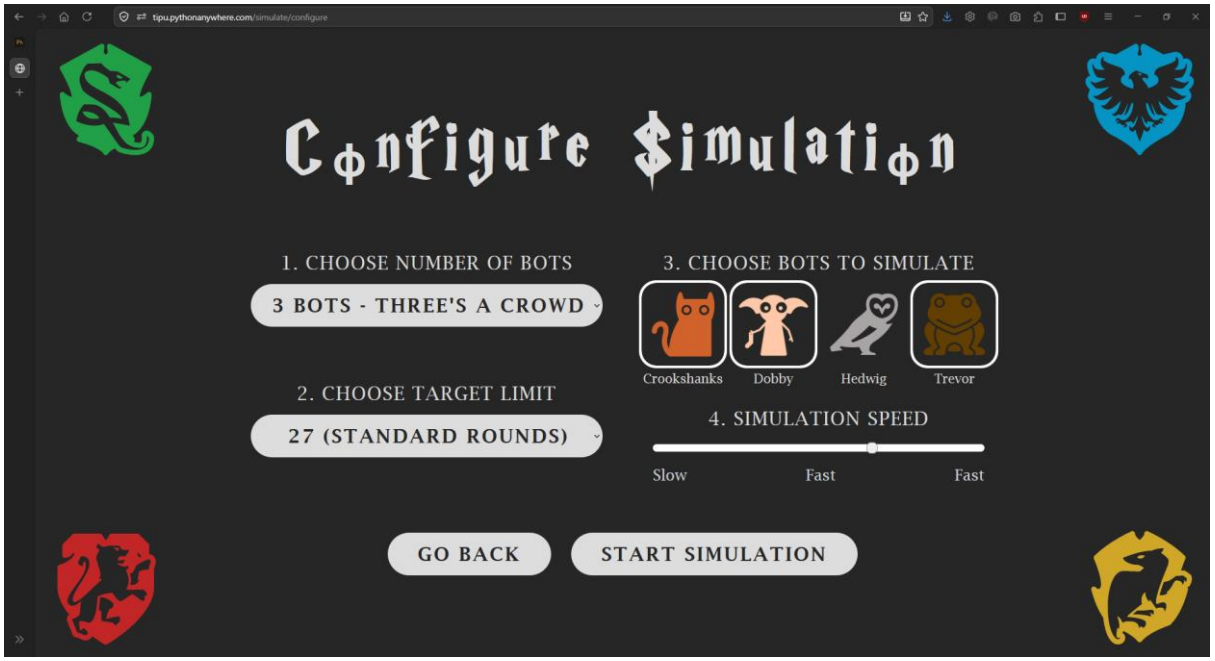

tipu.pythonanywhere.com/simulate/configure
Configure Simulation
1. CHOOSE NUMBER OF BOTS
3 BOTS - THREE'S A CROWD
2. CHOOSE TARGET LIMIT
27 (STANDARD ROUNDS)
3. CHOOSE BOTS TO SIMULATE
Crookshanks
Dobby
Hedwig
Trevor
4. SIMULATION SPEED
Slow
Fast
Fast
GO BACK
START SIMULATION

Fig.6

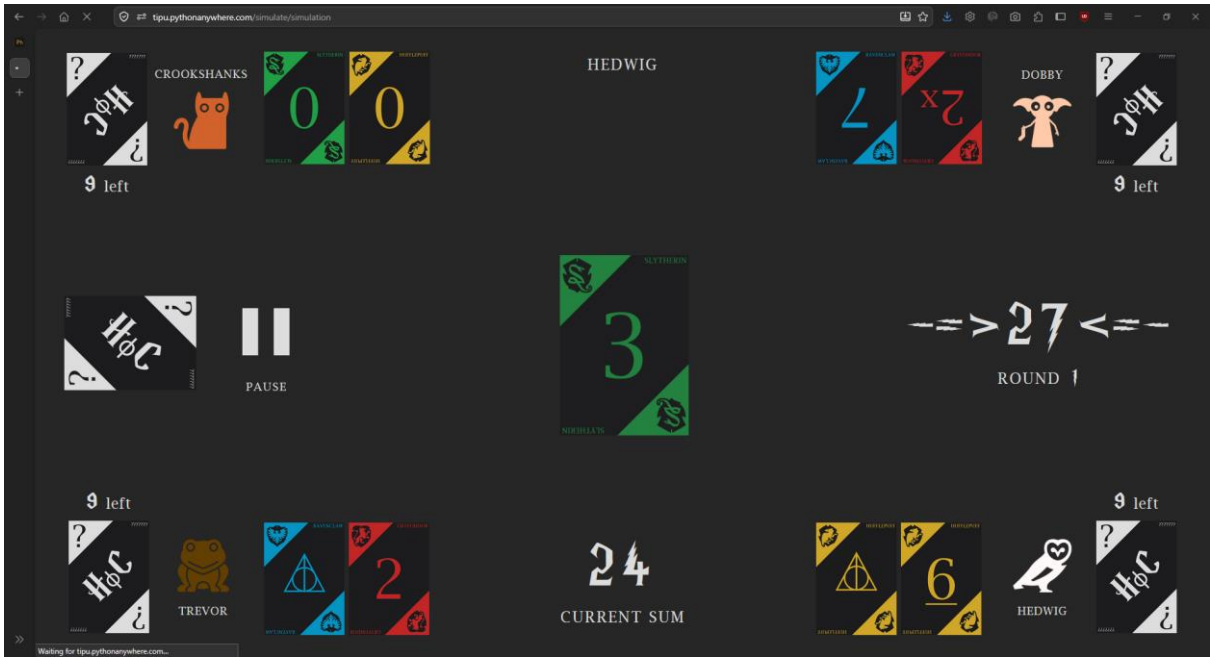

CROOKSHANKS
HEDWIG
DOBBY
9 left
9 left
PAUSE
ROUND 1
9 left
9 left
TREVOR
CURRENT SUM
HEDWIG

Fig.7

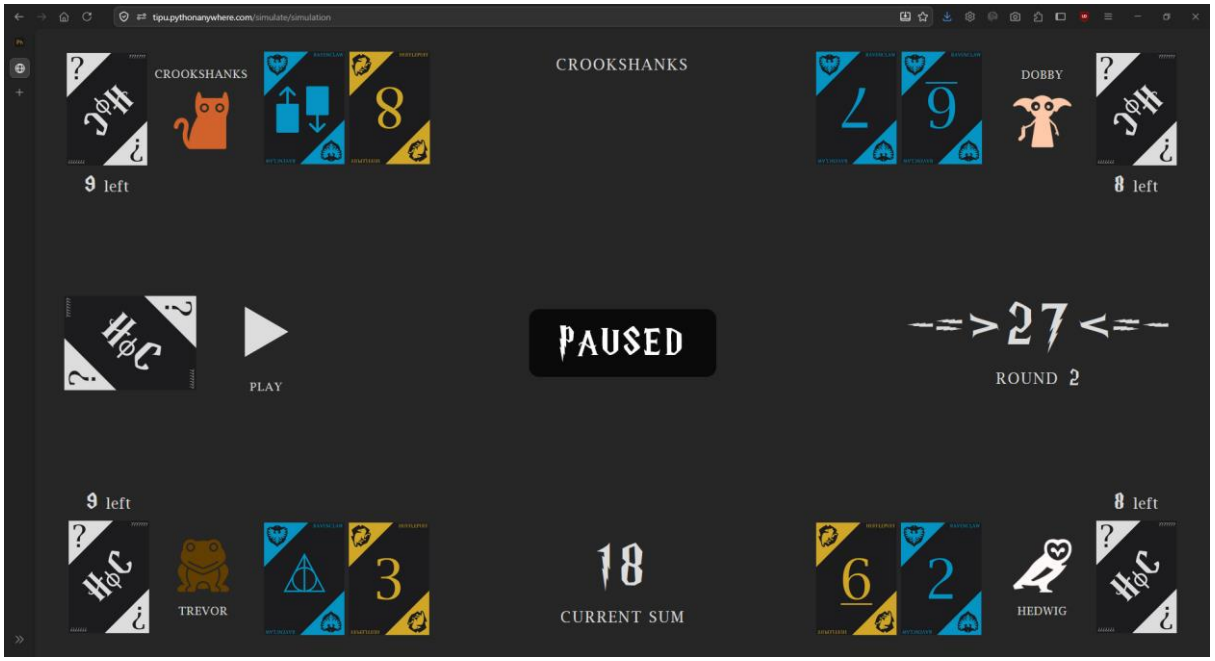

CROOKSHANKS
CROOKSHANKS
DOBBY
9 left
8 left
PLAY
PAUSED
ROUND 2
9 left
8 left
TREVOR
CURRENT SUM
HEDWIG

Fig.8

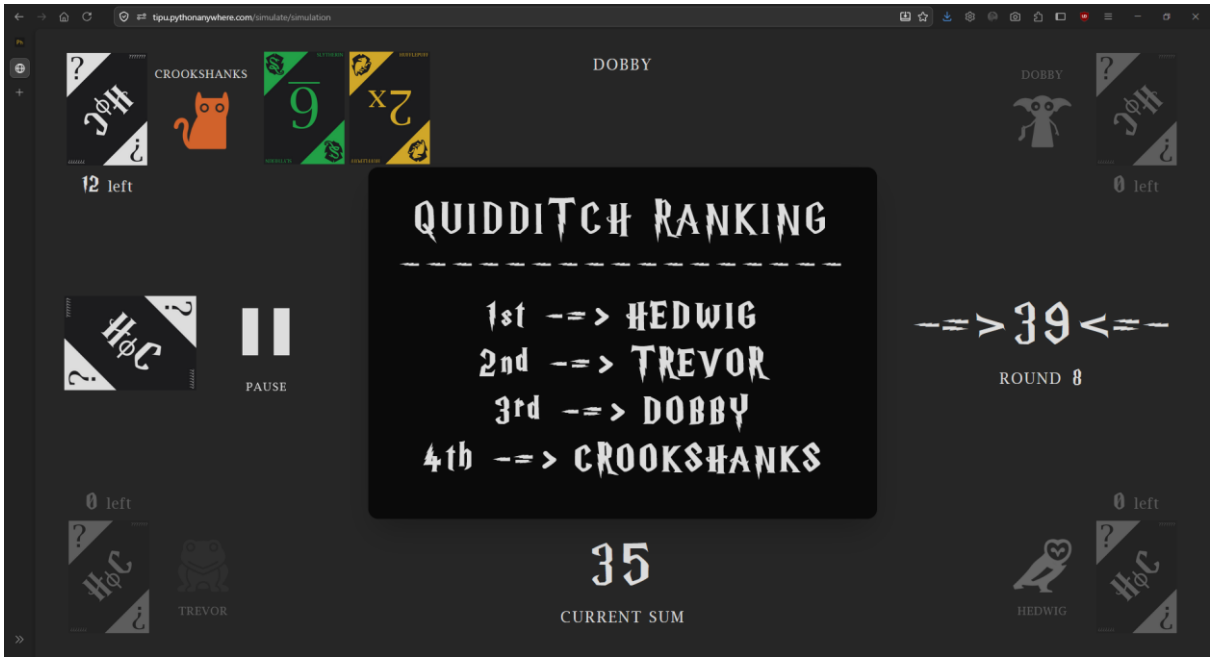

CROOKSHANKS
DOBBY
DOBBY
12 left
0 left
QUIDDITCH RANKING
1st -=> HEDWIG
2nd -=> TREVOR
3rd -=> DOBBY
4th -=> CROOKSHANKS
PAUSE
-=>39<=-
ROUND 8
0 left
0 left
TREVOR
35
CURRENT SUM
HEDWIG

Fig.9


PvE
PLAY AGAINST 1 BOT
PLAY AGAINST 2 BOTS
PLAY AGAINST 3 BOTS
GO BACK

Fig.10

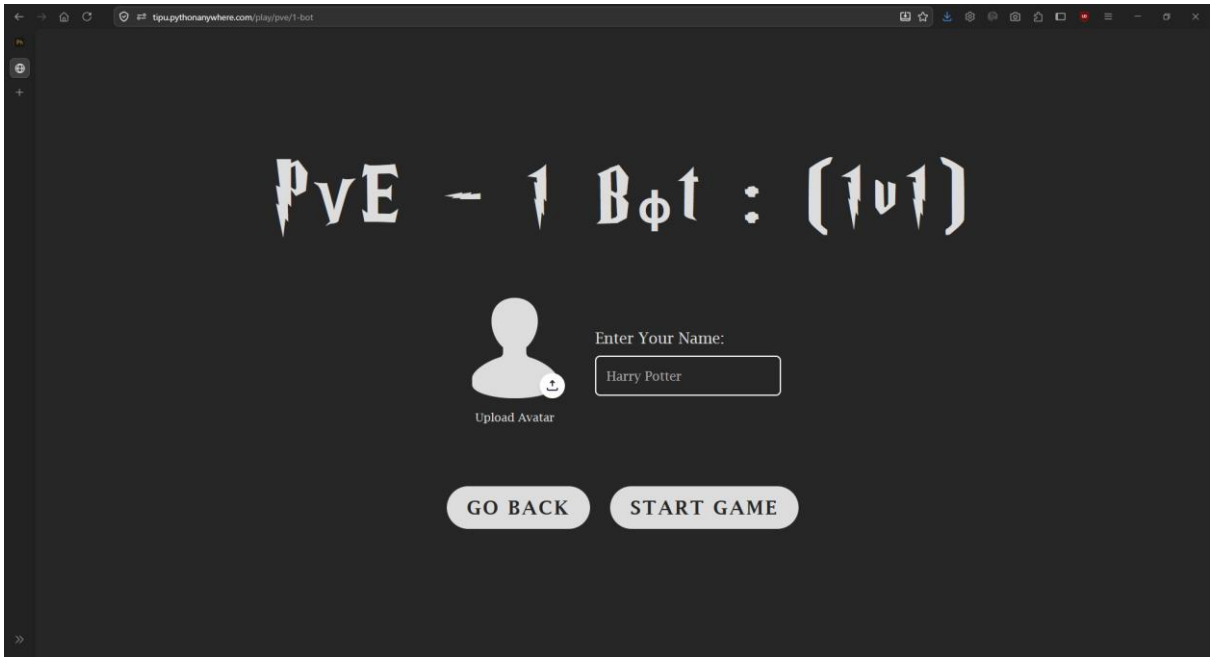

tipu.pythonanywhere.com/play/pve/1-bot
PvE - 1 Bot : (1v1)
Enter Your Name:
Harry Potter
Upload Avatar
GO BACK
START GAME

Fig.11

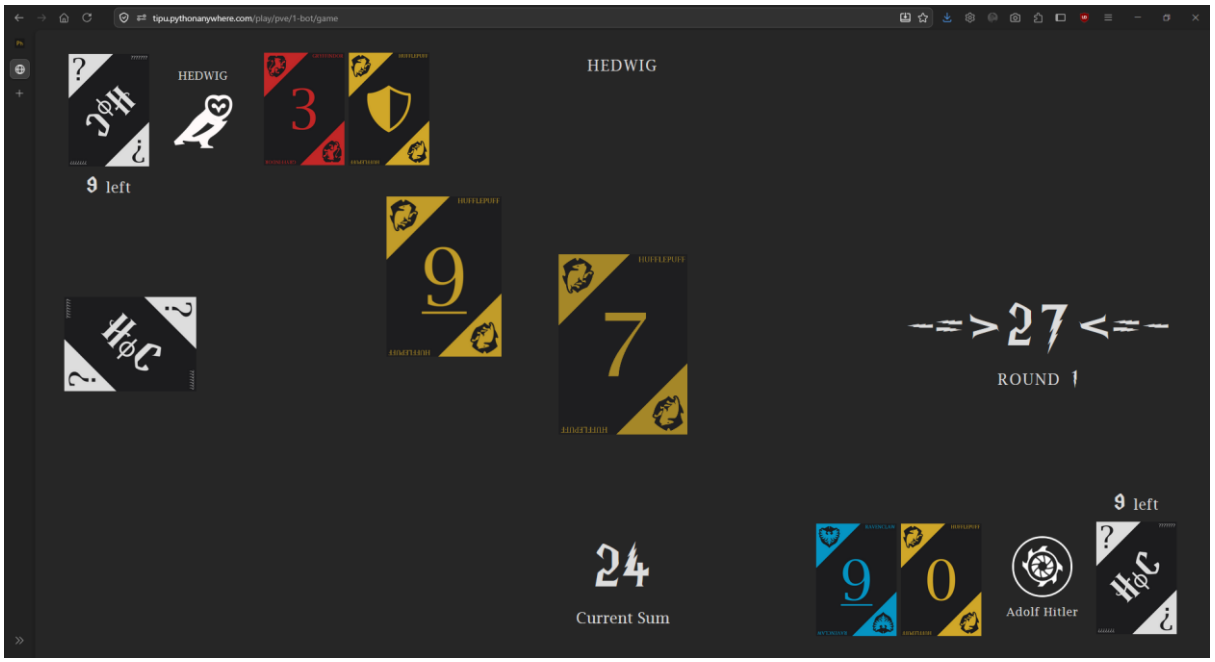
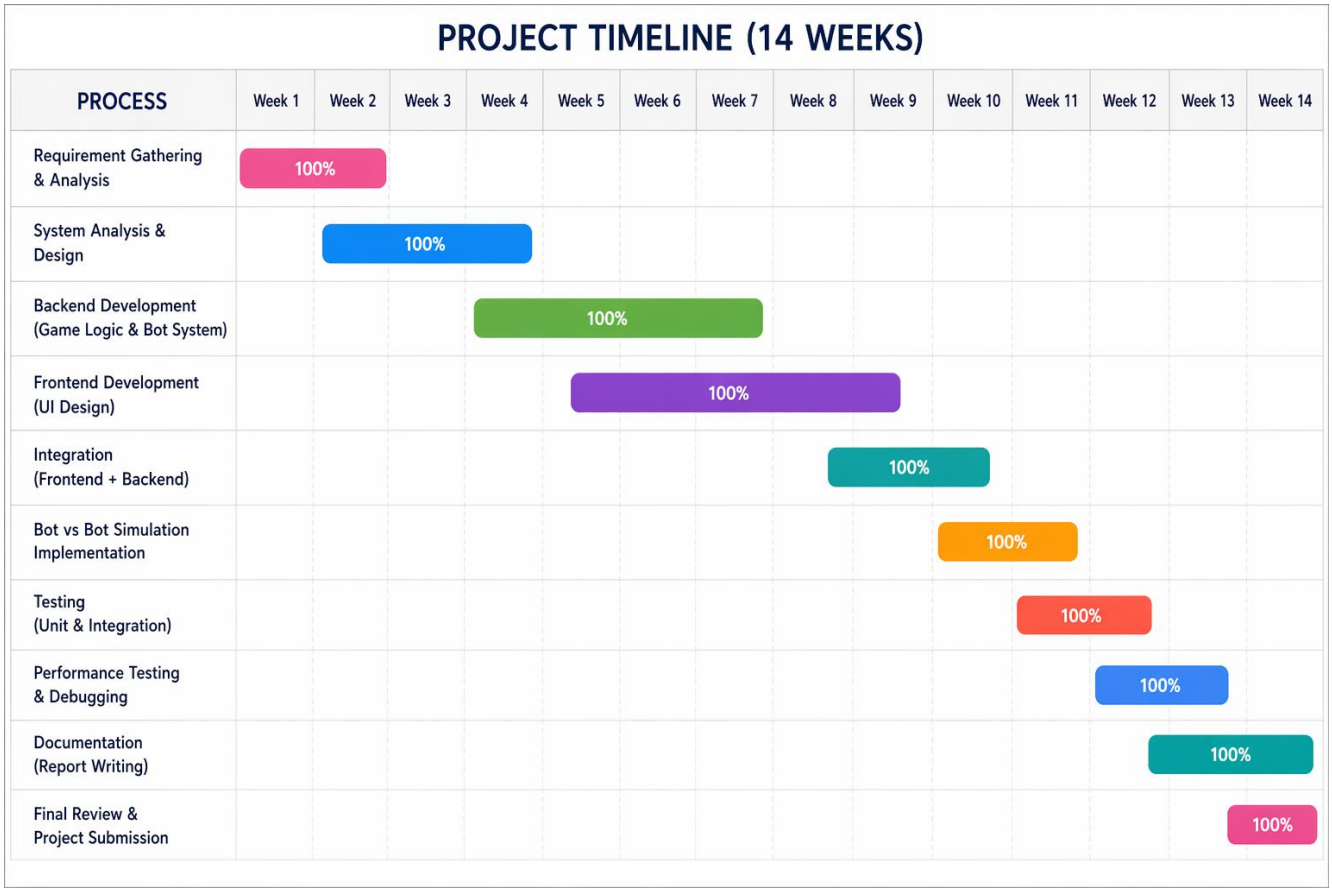

tipu.pythonanywhere.com/play/pve/1-bot/game
HEDWIG
9 left
HEDWIG
ROUND 1
24
Current Sum
9 left
Adolf Hitler

Fig.12

TIMELINE CHART



CONCLUSION

The **House of Cards** project successfully demonstrates the design and implementation of an interactive, strategy-based card game system integrated with intelligent bot behavior. The primary objective of developing a web-based card game that supports both player interaction and automated gameplay has been effectively achieved through a modular and structured approach.

The system incorporates a rule-based game engine that ensures smooth execution of game logic, including card distribution, turn management, score calculation, and win condition evaluation. The inclusion of AI-controlled bots enhances the overall functionality by enabling both **single-player (player vs bot)** and **automated simulation (bot vs bot)** modes. Unlike machine learning-based systems, the bots operate using a heuristic rule-based approach, making decisions based on the current game state. This ensures predictable, efficient, and stable gameplay while still maintaining a level of strategic challenge.

From a technical perspective, the integration of a web interface with a backend game engine demonstrates effective use of modern development practices. The system architecture is designed to be lightweight and scalable, even without reliance on complex databases or training models. The frontend ensures a user-friendly and interactive experience, while the backend handles all core computations and game logic efficiently.

The bot vs bot simulation feature plays a significant role in validating the correctness and robustness of the system. It allows continuous automated gameplay, which is useful for testing rule consistency, debugging, and analyzing different gameplay scenarios without human intervention. This contributes to improved system reliability and performance.

Overall, the project highlights the practical application of programming concepts such as control flow, data structures, modular design, and client-server interaction. It also demonstrates how artificial intelligence concepts can be implemented using simple rule-based logic to create engaging and intelligent systems.

FUTURE SCOPE

The House of Cards system has strong potential for further enhancement in terms of features, scalability, and user engagement. Some possible future improvements include:

- **Player vs Player (PvP) Mode:** Introduce real-time multiplayer functionality using technologies like WebSocket, allowing users to compete with each other over the internet with synchronized turns and live interaction.
- **Mobile Application Development:** Build a cross-platform mobile version using frameworks such as Flutter or React Native, enabling users to access the game on smartphones with touch-based controls and better accessibility.
- **Advanced AI Implementation:** Enhance the current rule-based bots by integrating machine learning or reinforcement learning techniques, allowing bots to adapt, learn from gameplay, and provide a more challenging experience.
- **Database Integration:** Incorporate a database system to manage user accounts, store game history, track scores, and implement leaderboards for performance comparison.
- **Enhanced UI/UX Design:** Improve the interface with animations, sound effects, and responsive layouts to create a more immersive and visually appealing gaming experience.
- **Multiplayer Rooms and Matchmaking:** Add functionality for users to create or join game rooms and implement matchmaking systems to connect players with similar skill levels.
- **In-Game Communication:** Introduce a chat feature to allow players to communicate during gameplay, enhancing social interaction.

REFERENCES

1. UNO Official / Game Background – https://www.unorules.com
2. Blackjack Game Overview – https://www.ebsco.com/research-starters/sports-and-leisure/blackjack
3. Three Card Poker – Rules and gameplay structure.
   Available at: https://www.pagat.com/poker/variants/3card.html/
4. Han, C., Peng, R., Zhao, H., 2023 – *Game theory: Optimum strategy for drawing cards in 21 points*
5. Mohd, T. K. et al., 2022 – *Implementing Playing Cards Blackjack Game using OpenCV*
6. Unity Technologies, *Unity Game Engine Documentation*. [Online]. Available: https://docs.unity.com
7. Flask Official Documentation. Available at: https://flask.palletsprojects.com/
8. Python Documentation. Available at: https://docs.python.org/3/
9. *Node.js Official Documentation.* Available at: https://nodejs.org/
10. Game Theory Concepts.
    Osborne, M. J. (2004). *An Introduction to Game Theory*. Oxford University Press.